

Universidad Nacional de Costa Rica
Escuela de Informática

Estructura de Datos

Sistema de Clustering Jerárquico
Dendrograma y Clasificación de Películas

Segundo Proyecto

Andrey Jesús Solano Rojas
Wilson José Umaña Ríos
Gabriel Sánchez Chacón

Profesor:
Prof. Steven Brenes Chavarría

Noviembre, 2025

Índice

1. Introducción	5
1.1. Objetivos del Proyecto	5
2. Marco Teórico	5
2.1. ¿Qué es un Dendrograma?	5
2.1.1. Componentes de un Dendrograma	5
2.2. Clustering Jerárquico Aglomerativo	6
2.2.1. Average Linkage	6
3. Técnicas de Normalización	6
3.1. Normalización Min-Max	6
3.2. Estandarización Z-Score	7
3.3. Transformación Logarítmica	7
4. Métricas de Distancia	7
4.1. Distancia Euclidiana	7
4.2. Distancia Manhattan	7
4.3. Distancia Coseno	7
4.4. Distancia de Hamming	7
5. Arquitectura del Software	8
5.1. Estructura de Carpetas	8
5.2. Flujo de Datos	9
6. Patrones de Diseño Implementados	9
6.1. Patrón MVC (Model-View-Controller)	9
6.2. Patrón Singleton	10
6.3. Patrón Iterator	10
6.4. Patrón Strategy	10
6.4.1. Estrategias de Distancia	11
6.4.2. Estrategias de Normalización	11
6.5. Patrón Factory Method	11
7. Estructuras de Datos Implementadas	12
7.1. CustomList (Lista Enlazada)	12
7.2. CustomVector (Arreglo Dinámico)	12
7.3. CustomMatrix (Matriz Bidimensional)	12
7.4. CustomHashMap (Tabla Hash)	13
8. Análisis de Complejidad Algorítmica	13
8.1. Parseo de CSV	13
8.2. Vectorización de Películas	13
8.3. Normalización del Dataset	14
8.4. Construcción de Matriz de Distancias	14
8.5. Algoritmo de Clustering Jerárquico	14

8.6. Average Linkage - Cálculo de Distancia	14
8.7. Exportación a JSON	15
9. Construcción del Dendrograma	15
9.1. Matriz de Distancias	15
9.2. Proceso de Fusión de Clusters	15
9.3. Average Linkage	16
9.4. Reconstrucción de la Matriz	16
10. Sistema de Configuración	16
10.1. ConfigurationManager (Singleton)	16
10.2. Aplicación de Pesos	17
10.3. Configuración de Variables Categóricas	17
11. Operaciones Vectoriales	17
11.1. Producto Punto	17
11.2. Norma Euclidiana	17
11.3. Resta Vectorial	17
12. Interfaz de Usuario	18
12.1. Diseño de la Interfaz	18
12.2. Flujo de Interacción	18
13. Formato de Exportación JSON	19
13.1. Estructura del JSON	19
13.2. Características del Formato	19
14. Implementación Detallada de Servicios	20
14.1. CSVParser	20
14.2. VectorizationService	20
14.3. CategoryIndexer	21
14.4. MovieVectorizer	21
14.5. DendogramBuilder	22
14.6. JSONExporter	23
15. Pruebas y Validación	24
15.1. Casos de Prueba	24
15.1.1. Prueba 1: Dataset Pequeño	24
15.1.2. Prueba 2: Comparación de Métricas	24
15.1.3. Prueba 3: Efecto de Ponderación	24
15.2. Validación de Resultados	25
16. Optimizaciones Implementadas	25
16.1. Vectorización One-Hot Eficiente	25
16.2. Matriz Simétrica	25
16.3. Crecimiento Exponencial de Vectores	25

17. Características de la Interfaz WPF	25
17.1. Diseño Moderno	25
17.2. Estilos Personalizados	26
17.3. Responsividad	26
18. Limitaciones y Trabajo Futuro	26
18.1. Limitaciones Actuales	26
18.2. Mejoras Propuestas	27
19. Conclusiones	27
19.1. Logros Principales	27
19.2. Aprendizajes Clave	28
19.3. Aplicabilidad	28
19.4. Reflexión Final	28
20. Referencias	29
A. Código Fuente Relevante	29
A.1. Clase Movie	29
A.2. Clase Cluster	30
B. Diagramas de Flujo	31
B.1. Flujo del Sistema Completo	31
C. Ejemplos de Uso	31
C.1. Ejemplo 1: Clustering Básico	31
C.2. Ejemplo 2: Énfasis en Géneros	31
C.3. Ejemplo 3: Análisis Temporal	32

1. Introducción

El análisis de grandes volúmenes de datos es fundamental en la era de la información. Una de las técnicas más poderosas para descubrir patrones ocultos y relaciones entre elementos es el clustering jerárquico, método que permite agrupar datos basándose en similitudes sin necesidad de conocer previamente el número de grupos.

Este proyecto implementa un sistema completo de clustering jerárquico aplicado al dominio cinematográfico, permitiendo descubrir patrones de similitud entre películas basándose en múltiples características como presupuesto, popularidad, géneros, reparto y más. La visualización de estos agrupamientos se realiza mediante un dendrograma, representación gráfica que muestra la estructura jerárquica de las relaciones.

1.1. Objetivos del Proyecto

- Implementar un sistema de clustering jerárquico desde cero
- Aplicar múltiples métricas de distancia y técnicas de normalización
- Desarrollar estructuras de datos personalizadas eficientes
- Implementar patrones de diseño de software robustos
- Generar dendrogramas exportables en formato JSON
- Crear una interfaz de usuario moderna y funcional en WPF

2. Marco Teórico

2.1. ¿Qué es un Dendrograma?

Un dendrograma es una representación gráfica en forma de árbol que ilustra la jerarquía de agrupamiento entre elementos de un conjunto de datos. Cada nivel del árbol representa una distancia o nivel de similitud al cual se fusionaron dos grupos.

2.1.1. Componentes de un Dendrograma

- **Hojas:** Representan elementos individuales del dataset
- **Nodos internos:** Representan la fusión de dos clusters
- **Altura:** Indica la distancia a la cual se fusionaron clusters
- **Ramas:** Conectan clusters fusionados

2.2. Clustering Jerárquico Aglomerativo

El método aglomerativo (bottom-up) sigue este proceso:

1. Inicializar cada elemento como un cluster individual
2. Calcular distancias entre todos los pares de clusters
3. Fusionar los dos clusters más cercanos
4. Actualizar matriz de distancias
5. Repetir hasta tener un único cluster

2.2.1. Average Linkage

El criterio Average Linkage calcula la distancia entre dos clusters como el promedio ponderado de las distancias:

$$d(C_{AB}, C_k) = \frac{|C_A| \cdot d(C_A, C_k) + |C_B| \cdot d(C_B, C_k)}{|C_A| + |C_B|} \quad (1)$$

donde $|C_A|$ y $|C_B|$ son los tamaños de los clusters A y B, y d es la distancia entre clusters.

3. Técnicas de Normalización

La normalización es crucial para asegurar que variables con diferentes escalas contribuyan equitativamente al análisis de clustering.

3.1. Normalización Min-Max

Transforma los valores a un rango específico, típicamente $[0, 1]$:

$$f(x) = \frac{x - \min(X)}{\max(X) - \min(X)} \quad (2)$$

Cuándo usar: Variables con la misma escala (edades, fechas, salarios).

Ventajas:

- Preserva la forma de la distribución original
- Fácil de interpretar
- Mantiene relaciones de distancia proporcionales

3.2. Estandarización Z-Score

Transforma los datos para tener media 0 y desviación estándar 1:

$$f(x) = \frac{x - \mu}{\sigma} \quad (3)$$

donde μ es la media y σ la desviación estándar.

Cuándo usar: Variables con diferentes unidades o escalas.

3.3. Transformación Logarítmica

Aplica logaritmo natural para reducir el impacto de valores extremos:

$$f(x) = \log(x + 1) \quad (4)$$

Cuándo usar: Datos muy sesgados o con valores extremos.

4. Métricas de Distancia

4.1. Distancia Euclidiana

Mide la distancia en línea recta entre dos puntos:

$$d(x, y) = \sqrt{\sum_{i=0}^n (x_i - y_i)^2} \quad (5)$$

4.2. Distancia Manhattan

Suma las diferencias absolutas en cada dimensión:

$$d(x, y) = \sum_{i=0}^n |x_i - y_i| \quad (6)$$

4.3. Distancia Coseno

Mide el ángulo entre dos vectores:

$$d(x, y) = 1 - \frac{x \cdot y}{||x|| \cdot ||y||} \quad (7)$$

4.4. Distancia de Hamming

Cuenta el número de posiciones donde vectores difieren:

$$d(x, y) = \sum_{i=0}^n \mathbb{I}_{x_i \neq y_i} \quad (8)$$

5. Arquitectura del Software

5.1. Estructura de Carpetas

El proyecto sigue una organización modular basada en responsabilidades:

```
Project02-Dendogram/
Models/                # Modelos de datos
    Movie.cs
    Cluster.cs
    ClusterJSON.cs
DataStructures/        # Estructuras personalizadas
    CustomList.cs
    CustomVector.cs
    CustomMatrix.cs
    CustomHashMap.cs
    Interfaces/
        IIterable.cs
        IIterator.cs
    Iterators/
        ListIterator.cs
        VectorIterator.cs
Services/              # Lógica de negocio
    CSVParser.cs
    VectorizationService.cs
    CategoryIndexer.cs
    MovieVectorizer.cs
    ConfigurationManager.cs
    DendogramBuilder.cs
    JSONExporter.cs
Strategies/            # Patrones Strategy
    Distance/
        IDistanceStrategy.cs
        EuclideanDistance.cs
        ManhattanDistance.cs
        CosineDistance.cs
        HammingDistance.cs
        DistanceFactory.cs
    Normalization/
        INormalizationStrategy.cs
        MinMaxNormalization.cs
        ZScoreNormalization.cs
        LogNormalization.cs
        NormalizationFactory.cs
Presentation/          # Capa de presentación (MVC)
    ClusteringJerarquico.xaml
    ClusteringJerarquico.xaml.cs
```



```
ClusteringModel.cs
ClusteringController.cs
Utils/                # Utilidades
    VectorOperations.cs
Resources/            # Recursos y datos
    dataset-movies.csv
```

5.2. Flujo de Datos

El sistema procesa los datos siguiendo un pipeline de transformación:

1. **Carga de Datos:** CSV → Lista de películas
2. **Vectorización:** Películas → Vectores numéricos
3. **Normalización:** Vectores → Vectores normalizados
4. **Ponderación:** Aplicación de pesos configurados
5. **Clustering:** Construcción de dendrograma
6. **Exportación:** Dendrograma → JSON

6. Patrones de Diseño Implementados

6.1. Patrón MVC (Model-View-Controller)

Separa la lógica de presentación, datos y control:

- **Model (ClusteringModel.cs):** Contiene el estado de la aplicación
- **View (ClusteringJerarquico.xaml):** Interfaz gráfica WPF
- **Controller (ClusteringController.cs):** Coordina operaciones

Beneficios:

- Separación clara de responsabilidades
- Facilita pruebas unitarias
- Permite cambios en la UI sin afectar lógica de negocio

6.2. Patrón Singleton

ConfigurationManager implementa Singleton para mantener configuración única:

```
1 internal class ConfigurationManager
2 {
3     private static ConfigurationManager _instance;
4
5     private ConfigurationManager()
6     {
7         InitializeDefaultConfiguration();
8     }
9
10    public static ConfigurationManager GetInstance()
11    {
12        if (_instance == null)
13            _instance = new ConfigurationManager();
14        return _instance;
15    }
16 }
```

Listing 1: Implementación del Singleton

Razón: Garantiza una única instancia de configuración en toda la aplicación.

6.3. Patrón Iterator

Permite recorrer colecciones sin exponer su estructura interna:

```
1 public interface IIterator<T>
2 {
3     bool HasNext();
4     T Next();
5     void Reset();
6 }
7
8 public interface IIterable<T>
9 {
10     IIterator<T> CreateIterator();
11 }
```

Listing 2: Interfaces del Iterator

Implementaciones:

- ListIterator: Recorre listas enlazadas
- VectorIterator: Recorre vectores dinámicos

6.4. Patrón Strategy

Encapsula algoritmos intercambiables en tiempo de ejecución.

6.4.1. Estrategias de Distancia

```
1 interface IDistanceStrategy
2 {
3     double Calculate(CustomVector<double> v1,
4                     CustomVector<double> v2);
5 }
6
7 class EuclideanDistance : IDistanceStrategy { ... }
8 class ManhattanDistance : IDistanceStrategy { ... }
9 class CosineDistance : IDistanceStrategy { ... }
10 class HammingDistance : IDistanceStrategy { ... }
```

Listing 3: Strategy para Distancias

6.4.2. Estrategias de Normalización

```
1 interface INormalizationStrategy
2 {
3     void CalculateStats(CustomList<Movie> movies,
4                         int featureIndex);
5     double Normalize(double value);
6 }
7
8 class MinMaxNormalization : INormalizationStrategy { ... }
9 class ZScoreNormalization : INormalizationStrategy { ... }
10 class LogNormalization : INormalizationStrategy { ... }
```

Listing 4: Strategy para Normalización

6.5. Patrón Factory Method

Centraliza la creación de objetos complejos:

```
1 class DistanceFactory
2 {
3     public static IDistanceStrategy CreateDistance(string type)
4     {
5         switch (type.ToLower())
6         {
7             case "euclidean":
8                 return new EuclideanDistance();
9             case "manhattan":
10                 return new ManhattanDistance();
11             case "cosine":
12                 return new CosineDistance();
13             case "hamming":
14                 return new HammingDistance();
15             default:
```

```
16         throw new ArgumentException("Tipo inválido");
17     }
18 }
19 }
```

Listing 5: Factory para Distancias

7. Estructuras de Datos Implementadas

7.1. CustomList (Lista Enlazada)

Propósito: Almacenar colecciones dinámicas de películas y clusters.

Estructura:

- Cada nodo contiene un dato y referencia al siguiente
- Mantiene referencias a cabeza y cola
- Contador de tamaño

Operaciones principales:

- Add(T item): $O(1)$ - Agregar al final
- GetAt(int index): $O(n)$ - Acceso por índice
- RemoveAt(int index): $O(n)$ - Eliminación

7.2. CustomVector (Arreglo Dinámico)

Propósito: Representar vectores de características de películas.

Estrategia de crecimiento: Duplica capacidad cuando está lleno.

Operaciones principales:

- Add(T item): $O(1)$ amortizado
- GetAt(int index): $O(1)$ - Acceso directo
- SetAt(int index, T value): $O(1)$

7.3. CustomMatrix (Matriz Bidimensional)

Propósito: Almacenar matriz de distancias entre películas.

Implementación: Array bidimensional $n \times n$ donde n es el número de películas.

Optimización: Solo se calcula y almacena la mitad superior de la matriz (simétrica).

7.4. CustomHashMap (Tabla Hash)

Propósito: Indexar categorías únicas (géneros, actores, directores).

Características:

- Resolución de colisiones por encadenamiento
- Factor de carga: 0.75
- Redimensionamiento automático

Operaciones:

- Put(TKey key, TValue value): $O(1)$ promedio
- Get(TKey key): $O(1)$ promedio
- ContainsKey(TKey key): $O(1)$ promedio

8. Análisis de Complejidad Algorítmica

8.1. Parseo de CSV

Complejidad: $O(n \cdot m)$

- n : Número de películas
- m : Número promedio de campos por película

Algorithm 1 Parseo de archivo CSV

```

1: Abrir archivo CSV
2: for cada línea del archivo do
3:   Separar campos por delimitador
4:   Crear objeto Movie
5:   Asignar valores a atributos
6:   Agregar película a lista
7: end for
8: return lista de películas

```

8.2. Vectorización de Películas

Complejidad: $O(n \cdot (k + \sum_{j=1}^c |C_j|))$

- n : Número de películas
- k : Características numéricas (constante = 7)
- c : Número de categorías (constante = 6)
- $|C_j|$: Tamaño del diccionario de categoría j

8.3. Normalización del Dataset

Complejidad: $O(n \cdot k)$

- n : Número de películas
- k : Número de características numéricas
- Dos pasadas sobre el dataset

8.4. Construcción de Matriz de Distancias

Complejidad: $O(n^2 \cdot d)$

- n : Número de películas
- d : Dimensionalidad de los vectores
- Calcula $\frac{n(n-1)}{2}$ distancias

8.5. Algoritmo de Clustering Jerárquico

Complejidad: $O(n^3)$

- Iteraciones: $n - 1$ fusiones
- Por iteración: búsqueda de mínimo $O(n^2)$ + actualización $O(n)$
- Complejidad total: $O(n \cdot n^2) = O(n^3)$

Optimización posible: Uso de heap para encontrar mínimo reduciría a $O(n^2 \log n)$.

Algorithm 2 Hierarchical Agglomerative Clustering

```

1: while número de clusters > 1 do
2:    $(i, j, dist) \leftarrow \text{encontrarClustersMasCercanos}(M)$ 
3:    $nuevoCluster \leftarrow \text{fusionar}(\text{cluster}[i], \text{cluster}[j], dist)$ 
4:   Actualizar matriz eliminando filas/columnas  $i, j$ 
5:   Calcular distancias a nuevo cluster (Average Linkage)
6:   Agregar nueva fila/columna para nuevoCluster
7:   Eliminar clusters  $i$  y  $j$  de lista
8:   Agregar nuevoCluster a lista
9: end while
10: return único cluster restante

```

8.6. Average Linkage - Cálculo de Distancia

Complejidad: $O(1)$ - Operación de tiempo constante.

Algorithm 3 Average Linkage - Distancia entre clusters

```

1:  $size_A \leftarrow$  número de elementos en A
2:  $size_B \leftarrow$  número de elementos en B
3:  $d_A \leftarrow M[A][k]$  ▷ Distancia A a k
4:  $d_B \leftarrow M[B][k]$  ▷ Distancia B a k
5:  $d_{AB,k} \leftarrow \frac{size_A \cdot d_A + size_B \cdot d_B}{size_A + size_B}$ 
6: return  $d_{AB,k}$ 

```

8.7. Exportación a JSON

Complejidad: $O(n)$

- Recorrido en profundidad del árbol
- Visita cada nodo exactamente una vez
- n nodos (hojas) $\Rightarrow 2n - 1$ nodos totales

9. Construcción del Dendrograma

9.1. Matriz de Distancias

La matriz de distancias es una estructura fundamental que almacena las distancias calculadas entre todos los pares de películas del dataset.

Propiedades:

- Matriz cuadrada de tamaño $n \times n$ donde n es el número de películas
- Simétrica: $M[i][j] = M[j][i]$
- Diagonal cero: $M[i][i] = 0$ (distancia de un elemento a sí mismo)

9.2. Proceso de Fusión de Clusters

El algoritmo mantiene una lista activa de clusters y una matriz de distancias que se actualiza en cada iteración:

1. **Inicialización:** Cada película es un cluster individual
2. **Búsqueda:** Encontrar el par de clusters (i, j) con menor distancia
3. **Fusión:** Crear nuevo cluster que contiene ambos clusters
4. **Actualización:**
 - Crear nueva matriz reducida
 - Calcular distancias del nuevo cluster usando Average Linkage
 - Copiar distancias entre clusters no fusionados
5. **Repetir:** Hasta que quede un solo cluster

9.3. Average Linkage

Ventajas:

- Menos sensible a outliers que Single Linkage
- Produce clusters más compactos que Complete Linkage
- Balance entre diferentes criterios de linkage

Interpretación: Los clusters más grandes tienen mayor peso en el cálculo de la nueva distancia.

9.4. Reconstrucción de la Matriz

En cada iteración, la matriz se reduce de tamaño $n \times n$ a $(n - 1) \times (n - 1)$:

Proceso:

1. La primera fila/columna corresponde al nuevo cluster fusionado
2. Las demás filas/columnas corresponden a clusters no fusionados
3. Se calculan distancias del nuevo cluster a todos los demás
4. Se copian distancias entre clusters existentes

Complejidad espacial: $O(n^3)$ acumulado durante todas las iteraciones.

10. Sistema de Configuración

10.1. ConfigurationManager (Singleton)

El ConfigurationManager centraliza toda la configuración del sistema mediante el patrón Singleton.

Responsabilidades:

- Gestión de pesos para cada característica
- Selección de estrategias de normalización
- Selección de métrica de distancia
- Habilitación/deshabilitación de variables

Vector de pesos: Mantiene un vector dinámico donde cada posición corresponde a una característica:

- Posiciones 0-6: Variables numéricas
- Posiciones 7+: Variables categóricas expandidas (one-hot)

10.2. Aplicación de Pesos

Después de la normalización, cada valor se multiplica por su peso:

$$v'_i = v_i \cdot w_i \quad (9)$$

donde v_i es el valor normalizado y w_i es el peso configurado.

Efecto: Permite controlar la importancia relativa de cada característica. Un peso de 0 elimina completamente la característica del análisis.

10.3. Configuración de Variables Categóricas

Para variables categóricas, un solo peso se aplica a todas las dimensiones del one-hot encoding correspondiente.

11. Operaciones Vectoriales

11.1. Producto Punto

Calcula la suma de productos elemento a elemento:

Algorithm 4 Producto punto de vectores

```

1: resultado  $\leftarrow$  0
2: for  $i \leftarrow 0$  to  $d - 1$  do
3:   resultado  $\leftarrow$  resultado +  $v_1[i] \cdot v_2[i]$ 
4: end for
5: return resultado

```

Complejidad: $O(d)$ donde d es la dimensión del vector.

11.2. Norma Euclidiana

Calcula la magnitud de un vector:

Algorithm 5 Norma euclidiana

```

1: suma  $\leftarrow$  0
2: for  $i \leftarrow 0$  to  $d - 1$  do
3:   suma  $\leftarrow$  suma +  $v[i]^2$ 
4: end for
5: return  $\sqrt{\textit{suma}}$ 

```

11.3. Resta Vectorial

Calcula la diferencia elemento a elemento:

Algorithm 6 Resta de vectores

```
1: Crear vector resultado de dimensión  $d$ 
2: for  $i \leftarrow 0$  to  $d - 1$  do
3:    $resultado[i] \leftarrow v_1[i] - v_2[i]$ 
4: end for
5: return resultado
```

12. Interfaz de Usuario

12.1. Diseño de la Interfaz

La interfaz gráfica está desarrollada en WPF (Windows Presentation Foundation) y sigue los principios de diseño moderno.

Componentes principales:

- **Panel de carga:** Selección de archivo CSV
- **Configuración de métrica:** ComboBox para elegir distancia
- **Variables numéricas:** 7 controles con:
 - CheckBox (habilitar/deshabilitar)
 - TextBox (peso)
 - ComboBox (estrategia de normalización)
- **Variables categóricas:** 6 controles con:
 - CheckBox (habilitar/deshabilitar)
 - TextBox (peso)
- **Botón de ejecución:** Inicia el proceso de clustering
- **Barra de estado:** Muestra progreso y mensajes

12.2. Flujo de Interacción

1. Usuario carga archivo CSV
2. Sistema valida y muestra información del dataset
3. Usuario configura:
 - Métrica de distancia
 - Normalización por variable
 - Pesos por variable
 - Variables activas
4. Usuario ejecuta clustering

5. Sistema procesa y muestra progreso
6. Sistema exporta resultado a JSON
7. Usuario recibe confirmación

13. Formato de Exportación JSON

13.1. Estructura del JSON

El dendrograma se exporta en formato JSON con la siguiente estructura:

```
1 {  
2   "n": "Titulo_de_pelicula_o_espacio",  
3   "d": 0.0,  
4   "c": [  
5     {  
6       "n": "pelicula_1",  
7       "d": 0.0,  
8       "c": []  
9     },  
10    {  
11      "n": "pelicula_2",  
12      "d": 0.0,  
13      "c": []  
14    }  
15  ]  
16 }
```

Listing 6: Estructura del JSON

Campos:

- **n:** Nombre (título de película para hojas, vacío para nodos internos)
- **d:** Distancia a la cual se fusionó este cluster
- **c:** Array de hijos (vacío para hojas)

13.2. Características del Formato

- **Jerárquico:** Refleja estructura de árbol
- **Recursivo:** Cada nodo tiene la misma estructura
- **Portable:** Estándar JSON compatible con cualquier visualizador
- **Legible:** Formato indentado para inspección manual

14. Implementación Detallada de Servicios

14.1. CSVParser

El servicio CSVParser es responsable de leer y parsear archivos CSV de películas.

Características principales:

- Utiliza `TextFieldParser` de `Microsoft.VisualBasic`
- Maneja campos encerrados en comillas
- Parseo robusto de valores numéricos y fechas
- Separación de valores categóricos por comas

```
1 public CustomList<Movie> ParseMovies()  
2 {  
3     CustomList<Movie> movies = new CustomList<Movie>();  
4  
5     using (TextFieldParser parser =  
6         new TextFieldParser(_filePath))  
7     {  
8         parser.TextFieldType = FieldType.Delimited;  
9         parser.SetDelimiters(",");  
10        parser.HasFieldsEnclosedInQuotes = true;  
11  
12        // Saltar encabezado  
13        if (!parser.EndOfData)  
14            parser.ReadLine();  
15  
16        while (!parser.EndOfData)  
17        {  
18            string[] fields = parser.ReadFields();  
19            Movie movie = ParseLine(fields);  
20            movies.Add(movie);  
21        }  
22    }  
23  
24    return movies;  
25 }
```

Listing 7: Método de parseo principal

14.2. VectorizationService

Coordina el proceso completo de vectorización de películas.

Responsabilidades:

- Construir índices de categorías únicas
- Vectorizar todas las películas del dataset

- Gestionar CategoryIndexer y MovieVectorizer

```
1 public void VectorizeMovies(CustomList<Movie> movies)
2 {
3     // Paso 1: construir indices
4     _indexer.BuildIndices(movies);
5
6     // Paso 2: vectorizar todas las películas
7     IIterator<Movie> it = movies.CreateIterator();
8
9     while (it.HasNext())
10    {
11        Movie movie = it.Next();
12        movie.FeatureVector = _vectorizer.Vectorize(movie);
13        movie.WeightedFeatureVector =
14            new CustomVector<double>(movie.FeatureVector);
15    }
16 }
```

Listing 8: Vectorización de películas

14.3. CategoryIndexer

Construye diccionarios que mapean valores categóricos únicos a índices numéricos para la codificación one-hot.

Proceso:

1. Recorre todas las películas
2. Para cada categoría (géneros, cast, etc.)
3. Registra valores únicos con un índice incremental
4. Mantiene un HashMap por cada categoría

14.4. MovieVectorizer

Convierte una película individual en un vector numérico.

Estructura del vector:

1. **Posiciones 0-6:** Variables numéricas
 - budget, popularity, revenue, runtime
 - voteaverage, votecount, releaseyear
2. **Posiciones 7+:** Variables categóricas (one-hot)
 - genres, cast, directors, keywords
 - companies, languages

```
1 private CustomVector<double> CreateOneHot(  
2     string[] items,  
3     CustomHashMap<string, int> hashmap)  
4 {  
5     CustomVector<double> oneHot = new CustomVector<double>();  
6  
7     // Inicializar con ceros  
8     for (int i = 0; i < hashmap.Count; i++)  
9         oneHot.Add(0);  
10  
11     if (items == null) return oneHot;  
12  
13     // Marcar posiciones correspondientes  
14     foreach (string item in items)  
15     {  
16         if (hashmap.ContainsKey(item))  
17             oneHot.SetAt(hashmap.Get(item), 1);  
18     }  
19  
20     return oneHot;  
21 }
```

Listing 9: Creación de vector one-hot

14.5. DendogramBuilder

Implementa el algoritmo de clustering jerárquico aglomerativo con Average Linkage.

Pasos principales:

1. Construir matriz de distancias inicial
2. Inicializar clusters individuales
3. Bucle principal:
 - Encontrar par más cercano
 - Fusionar clusters
 - Actualizar matriz de distancias
 - Actualizar lista de clusters
4. Retornar cluster raíz

```
1 private CustomMatrix<double> UpdateMatrix(  
2     CustomMatrix<double> oldMatrix,  
3     CustomList<Cluster> clusters,  
4     int keep, int drop)  
5 {  
6     int oldN = clusters.Count;
```

```

7      int newN = oldN - 1;
8      var newMatrix = new CustomMatrix<double>(newN);
9
10     int sizeKeep = clusters.GetAt(keep).Indexes.Count;
11     int sizeDrop = clusters.GetAt(drop).Indexes.Count;
12     int totalSize = sizeKeep + sizeDrop;
13
14     // Average Linkage: promedio ponderado
15     for (int i = 0; i < oldN; i++)
16     {
17         if (i == keep || i == drop) continue;
18
19         double distKeep = oldMatrix.GetAt(keep, i);
20         double distDrop = oldMatrix.GetAt(drop, i);
21
22         double newDist =
23             (sizeKeep * distKeep + sizeDrop * distDrop)
24             / totalSize;
25
26         // Asignar nueva distancia
27         newMatrix.SetAt(mergedIdx, ni, newDist);
28         newMatrix.SetAt(ni, mergedIdx, newDist);
29     }
30
31     return newMatrix;
32 }

```

Listing 10: Actualización de matriz con Average Linkage

14.6. JSONExporter

Convierte el árbol de clusters en formato JSON y lo exporta a archivo.

```

1  public ClusterJson ConvertToJson(Cluster cluster)
2  {
3      if (cluster == null) return null;
4
5      ClusterJson json = new ClusterJson();
6
7      // Distancia
8      json.d = cluster.Distance;
9
10     // Nombre (solo hojas tienen Movie)
11     json.n = (cluster.IsLeaf)
12         ? cluster.Movies.GetAt(0).Title
13         : "□";
14
15     // Hijos recursivos
16     if (!cluster.IsLeaf)
17     {

```

```
18         json.c.Add(ConvertToJson(cluster.Left));
19         json.c.Add(ConvertToJson(cluster.Right));
20     }
21
22     return json;
23 }
```

Listing 11: Conversión a JSON

15. Pruebas y Validación

15.1. Casos de Prueba

15.1.1. Prueba 1: Dataset Pequeño

Objetivo: Verificar correctitud básica del algoritmo.

Configuración:

- 5 películas
- Distancia euclidiana
- Normalización MinMax
- Todos los pesos = 1.0

Resultado esperado: Dendrograma con 4 fusiones, altura creciente.

15.1.2. Prueba 2: Comparación de Métricas

Objetivo: Verificar que diferentes métricas producen resultados diferentes.

Configuración:

- Mismo dataset
- 4 métricas distintas
- Misma configuración de pesos

Resultado esperado: Dendrogramas con diferentes estructuras y alturas.

15.1.3. Prueba 3: Efecto de Ponderación

Objetivo: Verificar que los pesos afectan el clustering.

Configuración:

- Dataset de películas
- Peso alto para género (15.0)
- Peso bajo para otras variables (1.0)

Resultado esperado: Películas del mismo género se agrupan primero.

15.2. Validación de Resultados

Criterios de validación:

1. Todas las películas aparecen en el dendrograma
2. No hay duplicados
3. Distancias son no negativas
4. Distancias aumentan con la altura del árbol
5. JSON es bien formado y parseable

16. Optimizaciones Implementadas

16.1. Vectorización One-Hot Eficiente

En lugar de crear vectores completos para cada categoría por separado, se construye el vector completo en una sola pasada.

Beneficio: Reduce creaciones de objetos y mejora localidad de caché.

16.2. Matriz Simétrica

Solo se calcula la mitad superior de la matriz de distancias, aprovechando la simetría:

Ahorro: $\frac{n(n-1)}{2}$ cálculos en lugar de n^2 .

16.3. Crecimiento Exponencial de Vectores

Los vectores dinámicos duplican su capacidad en lugar de crecer linealmente.

Beneficio: Amortiza el costo de redimensionamiento a $O(1)$ por inserción.

17. Características de la Interfaz WPF

17.1. Diseño Moderno

La interfaz utiliza estilos personalizados con:

- **Colores modernos:** Paleta basada en Tailwind CSS
- **Bordes redondeados:** CornerRadius en controles
- **Sombras sutiles:** Efectos de profundidad
- **Transiciones suaves:** Triggers para estados hover
- **Tipografía clara:** Jerarquía visual bien definida

17.2. Estilos Personalizados

```
1 <Style x:Key="ModernGroupBox" TargetType="GroupBox">
2   <Setter Property="Background" Value="White"/>
3   <Setter Property="BorderBrush" Value="#E1E8ED"/>
4   <Setter Property="BorderThickness" Value="1"/>
5   <Setter Property="Template">
6     <Setter.Value>
7       <ControlTemplate TargetType="GroupBox">
8         <Border Background="{TemplateBinding Background}"
9           BorderBrush="{TemplateBinding BorderBrush}"
10          BorderThickness="{TemplateBinding
11            BorderThickness}"
12          CornerRadius="8">
13           <!-- Contenido del template -->
14         </Border>
15       </ControlTemplate>
16     </Setter.Value>
17   </Setter>
</Style>
```

Listing 12: Estilo de GroupBox moderno

17.3. Responsividad

La interfaz se adapta a diferentes tamaños de ventana mediante:

- ScrollViewer para contenido largo
- Grid con proporciones dinámicas
- Controles que se redimensionan automáticamente

18. Limitaciones y Trabajo Futuro

18.1. Limitaciones Actuales

1. **Complejidad temporal:** $O(n^3)$ limita el tamaño del dataset
2. **Memoria:** Almacenamiento completo de matriz de distancias
3. **Visualización:** No hay visor integrado del dendrograma
4. **Tiempo de desarrollo:** Gran carga académica de otros cursos

18.2. Mejoras Propuestas

1. Optimización algorítmica:

- Implementar heap para reducir a $O(n^2 \log n)$
- Matriz triangular para reducir uso de memoria

2. Visualización integrada:

- Renderizado del dendrograma en WPF
- Interactividad: zoom, pan, tooltip
- Coloreado de clusters

3. Funcionalidades adicionales:

- Guardado/carga de configuraciones
- Comparación de múltiples dendrogramas
- Exportación a otros formatos (PNG, SVG)
- Estadísticas de clusters

4. Rendimiento:

- Procesamiento paralelo de distancias
- Barra de progreso más detallada
- Cancelación de operaciones largas

19. Conclusiones

Este proyecto ha implementado exitosamente un sistema completo de clustering jerárquico aplicado al análisis de películas, cumpliendo con todos los requisitos establecidos:

19.1. Logros Principales

1. **Estructuras de datos propias:** Se implementaron desde cero listas enlazadas, vectores dinámicos, matrices y tablas hash, garantizando control total sobre el comportamiento y rendimiento.
2. **Patrones de diseño:** La aplicación de MVC, Singleton, Strategy, Iterator y Factory Method resultó en un código modular, mantenible y extensible.
3. **Flexibilidad:** El sistema permite configurar métricas de distancia, estrategias de normalización y ponderación de variables, adaptándose a diferentes necesidades de análisis.
4. **Escalabilidad del diseño:** Aunque la complejidad algorítmica es $O(n^3)$, la arquitectura permite mejoras futuras sin cambios significativos.

5. **Exportación estándar:** El formato JSON facilita la integración con herramientas de visualización externas.
6. **Interfaz moderna:** WPF con diseño contemporáneo que mejora la experiencia del usuario.

19.2. Aprendizajes Clave

- La importancia de la normalización para evitar dominancia de variables de gran escala
- El impacto de la elección de métrica de distancia en los resultados
- La relevancia de patrones de diseño en sistemas complejos
- El balance entre complejidad algorítmica y claridad de código
- La necesidad de estructuras de datos eficientes en aplicaciones de ciencia de datos
- El valor de una interfaz de usuario bien diseñada para la usabilidad

19.3. Aplicabilidad

El sistema desarrollado es aplicable no solo a películas, sino a cualquier dominio que requiera análisis de similitud:

- **Recomendación de productos** en e-commerce
- **Análisis de perfiles de clientes** en marketing
- **Clasificación de documentos** en minería de texto
- **Análisis de secuencias genéticas** en bioinformática
- **Detección de anomalías** en seguridad informática
- **Agrupación de imágenes** en visión por computadora
- **Segmentación de redes sociales** en análisis de grafos

19.4. Reflexión Final

El desarrollo de este proyecto representó un desafío significativo que requirió la integración de conocimientos de algoritmos, estructuras de datos, patrones de diseño y desarrollo de interfaces gráficas. A pesar de las limitaciones de tiempo debido a la carga académica, se logró implementar un sistema funcional y robusto que cumple con los objetivos planteados.

La experiencia adquirida en la implementación de estructuras de datos personalizadas y la aplicación de patrones de diseño será invaluable para futuros proyectos de desarrollo de software. Además, el conocimiento profundo del algoritmo de clustering jerárquico y sus variantes proporciona una base sólida para incursionar en temas más avanzados de aprendizaje automático y análisis de datos.

20. Referencias

Referencias

- [1] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media.
- [2] Jain, A. K., & Dubes, R. C. (1988). *Algorithms for clustering data*. Prentice-Hall, Inc.
- [3] Kaufman, L., & Rousseeuw, P. J. (2009). *Finding groups in data: an introduction to cluster analysis*. John Wiley & Sons.
- [4] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- [5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms*. MIT press.
- [6] Microsoft Corporation. (2023). *Windows Presentation Foundation Documentation*. <https://docs.microsoft.com/en-us/dotnet/desktop/wpf/>
- [7] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- [8] Ward Jr, J. H. (1963). *Hierarchical grouping to optimize an objective function*. Journal of the American statistical association, 58(301), 236-244.
- [9] Müllner, D. (2013). *fastcluster: Fast hierarchical, agglomerative clustering routines for R and Python*. Journal of Statistical Software, 53(9), 1-18.

A. Código Fuente Relevante

A.1. Clase Movie

```
1 public class Movie
2 {
3     // Datos de Identificacion
4     public string Title { get; set; }
5
6     // Variables Numericas
7     public double Budget { get; set; }
8     public double Revenue { get; set; }
9     public double Runtime { get; set; }
10    public double Popularity { get; set; }
11    public double VoteAverage { get; set; }
12    public double VoteCount { get; set; }
13    public double ReleaseYear { get; set; }
14
15    // Variables Categorias
```

```

16     public string[] Genres { get; set; }
17     public string[] Cast { get; set; }
18     public string[] Directors { get; set; }
19     public string[] Keywords { get; set; }
20     public string[] ProductionCompanies { get; set; }
21     public string[] SpokenLanguages { get; set; }
22
23     // Vectores de características
24     public CustomVector<double> FeatureVector { get; set; }
25     public CustomVector<double> WeightedFeatureVector { get; set; }
26     }
27
28     public Movie()
29     {
30         FeatureVector = new CustomVector<double>();
31         WeightedFeatureVector = new CustomVector<double>();
32     }

```

Listing 13: Modelo de datos Movie

A.2. Clase Cluster

```

1  internal class Cluster
2  {
3      public CustomList<Movie> Movies { get; set; }
4      public CustomList<int> Indexes { get; set; }
5      public Cluster Left { get; set; }
6      public Cluster Right { get; set; }
7      public double Distance { get; set; }
8
9      public bool IsLeaf => Left == null && Right == null;
10
11     // Constructor para hoja
12     public Cluster(Movie movie, int index)
13     {
14         Movies = new CustomList<Movie>();
15         Movies.Add(movie);
16
17         Indexes = new CustomList<int>();
18         Indexes.Add(index);
19
20         Left = null;
21         Right = null;
22     }
23
24     // Constructor para nodo interno
25     public Cluster(Cluster A, Cluster B, double distance)
26     {

```

```
27     Left = A;
28     Right = B;
29     Distance = distance;
30
31     Movies = new CustomList<Movie>();
32     Movies.Copy(A.Movies);
33     Movies.Copy(B.Movies);
34
35     Indexes = new CustomList<int>();
36     Indexes.Copy(A.Indexes);
37     Indexes.Copy(B.Indexes);
38 }
39 }
```

Listing 14: Nodo del dendrograma

B. Diagramas de Flujo

B.1. Flujo del Sistema Completo

[node distance=2cm]

Nota: Los diagramas de flujo detallados están disponibles en la documentación digital del proyecto.

C. Ejemplos de Uso

C.1. Ejemplo 1: Clustering Básico

Configuración:

- Métrica: Euclidiana
- Normalización: MinMax para todas las variables
- Pesos: 1.0 para todas las variables

Resultado: Dendrograma que agrupa películas principalmente por presupuesto y popularidad.

C.2. Ejemplo 2: Énfasis en Géneros

Configuración:

- Métrica: Coseno
- Peso de géneros: 15.0
- Peso de otras variables: 1.0

Resultado: Películas del mismo género se agrupan primero, independientemente de otras características.

C.3. Ejemplo 3: Análisis Temporal

Configuración:

- Métrica: Manhattan
- Peso de ReleaseYear: 20.0
- Otras variables deshabilitadas

Resultado: Clustering puramente cronológico de películas.